

Stochastic Methods for Industrial Engineers - Project

Markov Chains (a, b, d) + Martingales (a.a-a.b, b) · 120 pts

1 Markov Chains

1.1 Markov Modeling (20 pts)

Process : the number of customers in a small single-server queue with finite capacity K (e.g. a coffee-shop counter like Netot or ATM), observed at equally spaced discrete time slots. State = # customers in $\{0, \dots, K\}$. Each slot, at most one event occurs: an arrival with probability λ (blocked if the buffer is full), or, if occupied, a service completion with probability μ - mutually exclusive, so $\lambda + \mu \leq 1$, giving $P(n \rightarrow n+1) = \lambda$ and $P(n \rightarrow n-1) = \mu$ directly. A classic birth-death chain - the standard queueing-theory / capacity-planning model. Illustrative constants used: $\lambda = 0.4$, $\mu = 0.5$, $K = 7$ (chosen for exposition, not fit to data).

(We decided to make the queue have a finite capacity only to make this random process realistic, and so it will be meaningful for sub-section a.b.)

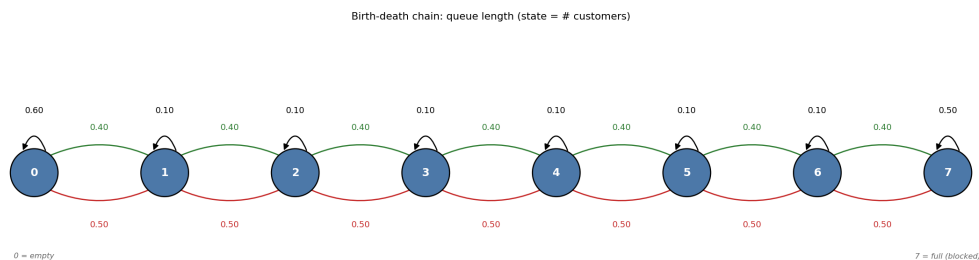


Fig 1. State-transition diagram: green = arrival (birth), red = departure (death), black loops = self-transitions.

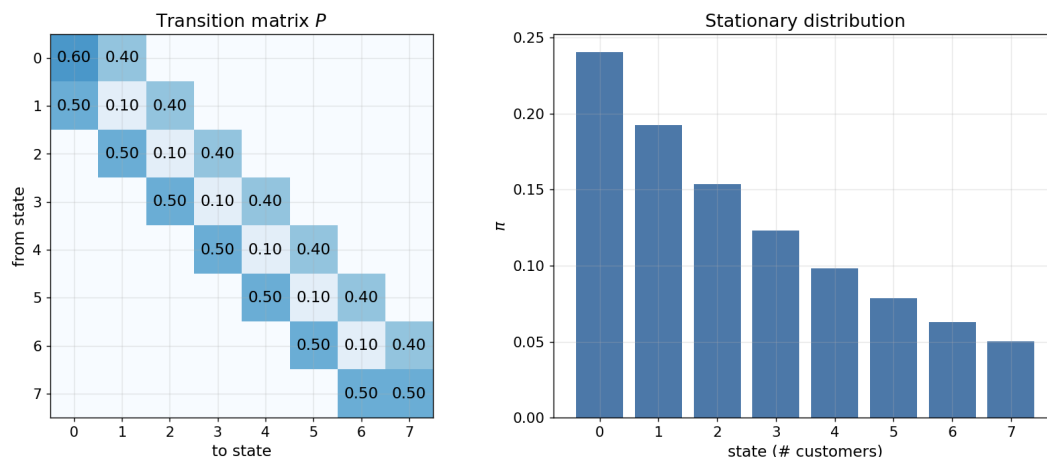


Fig 2. Transition matrix P (left) and stationary distribution π (right).

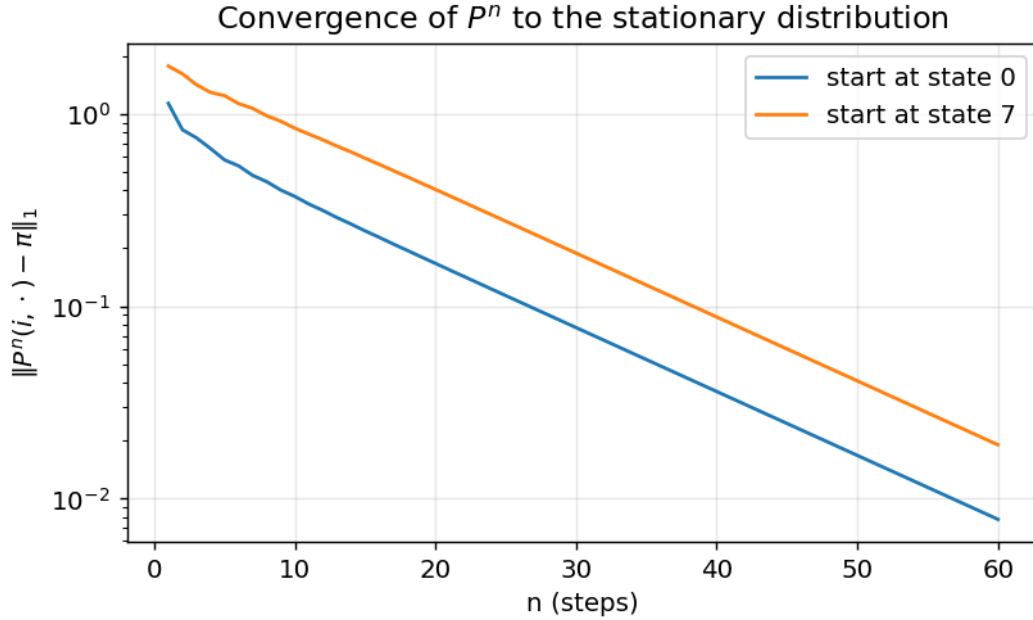


Fig 3. Convergence of $P^n(i, \cdot)$ to π from two different starting states (evidence for ergodicity).

1.1.1 Why is it a good or bad model

Why it is a good Markov chain model:

1. The state space is discrete and naturally ordered, queue length 0, 1, 2, and so on, fitting a birth-death chain structure.
2. The current queue length is a sufficient statistic for the next transition, you do not need the full arrival history to predict what happens next, which is the memoryless property working as intended.
3. Poisson type arrivals are a standard and reasonable approximation in many real queueing systems, here specifically the M/M/1 single server case.

Why it is a bad Markov chain model:

1. Arrival rates are usually time inhomogeneous. A constant λ ignores lunch rush versus quiet hours, so you need $\lambda(t)$ rather than one fixed rate.
2. Service times are often not memoryless. A human barista's remaining service time typically depends on elapsed service time already spent (drink complexity, learning curve, fatigue), which violates the exponential service assumption and calls for a semi Markov or phase type service model instead.
3. The model ignores batching. Customers often arrive in groups (couples, coworkers), not one at a time, which the simple birth-death chain does not capture.
4. Real customers may refuse to join a long queue if it looks too long (balking) or leave after waiting too long (reneging), both of which are state and time dependent behaviors outside the basic chain, which the markov chain ignore.

1.1.2 Chain properties

- Irreducible? Yes - from any state n , state 0 is reachable via successive departure-only steps (prob $\mu > 0$), and state K is reachable from 0 via successive arrival-only steps (prob $\lambda > 0$); all states communicate in one class.
- Aperiodic? Yes - state 0 has a self-loop, $P(0,0) = 1 - \lambda > 0$. For an irreducible chain the period is common to all states and divides the length of every cycle through any one state; a length-1 cycle forces period 1.
- Positive recurrent? Yes - a finite, irreducible chain is always positive recurrent: the stationary distribution π exists with $\pi_i > 0$ for every i , and the mean return time to i equals $1/\pi_i < \infty$ (verified numerically - see Fig 2).
- Ergodic? Yes - finite + irreducible + aperiodic $\Rightarrow$ ergodic: a unique stationary distribution exists and $P^n(i, \cdot) \rightarrow \pi$ for every starting state i , exactly as shown in Fig 3.
- Absorbing? No - every state has self-loop probability < 1 (since $0 < \lambda, \mu < 1$), so there is no trapping state; consistent with irreducibility (a chain with > 1 state cannot be both irreducible and absorbing).
- Assumptions: time-homogeneous λ, μ ; at most one event (an arrival or a service completion, never both) per slot, requiring $\lambda + \mu \leq 1$; finite buffer K with blocking rather than an unbounded queue.

1.2 Hidden Markov Model (25 pts)

Data is simulated directly from the Q1 queueing chain (same λ, μ, K , and P as part a). The true queue length is hidden; we observe a noisy sensor reading $Y_t = X_t + \varepsilon_t$, $\varepsilon_t \sim \mathcal{N}(0, \sigma^2)$, $\sigma = 0.1$. We simulate 6,000 steps (4,800 train / 1,200 test). An 8-state Gaussian HMM (matching $K + 1$) is fit by Baum-Welch (EM) to recover the hidden queue length - using the best of 20 random restarts by training log-likelihood, since a single restart is unreliable here (see b.c). Because the data is simulated, the recovered states can be scored against ground truth.

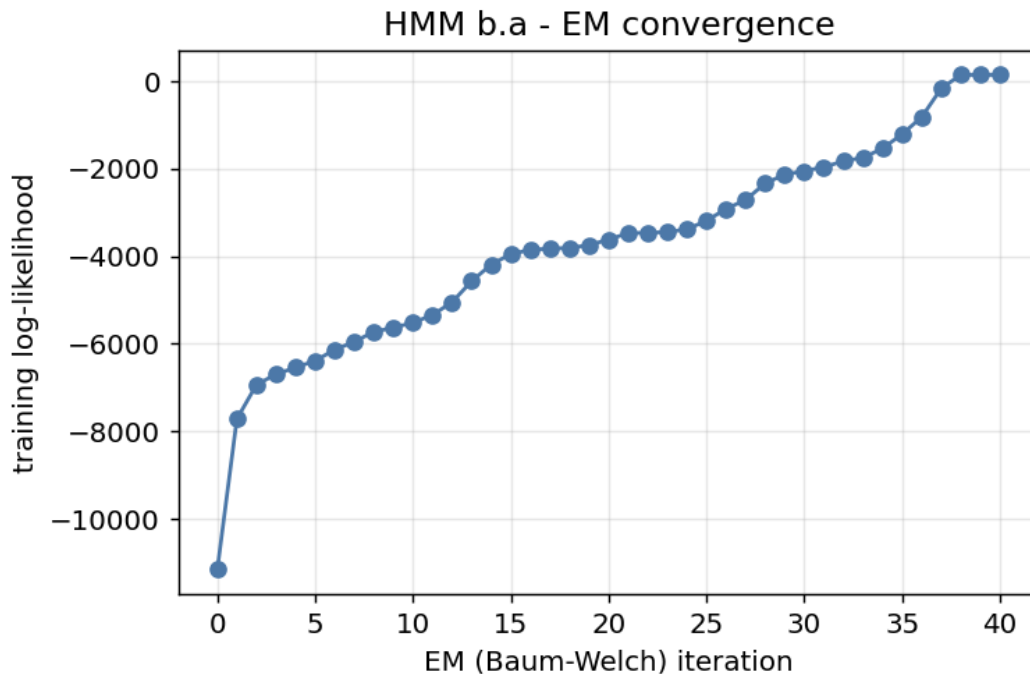


Fig 4 (b.a). EM log-likelihood increases monotonically to a plateau.

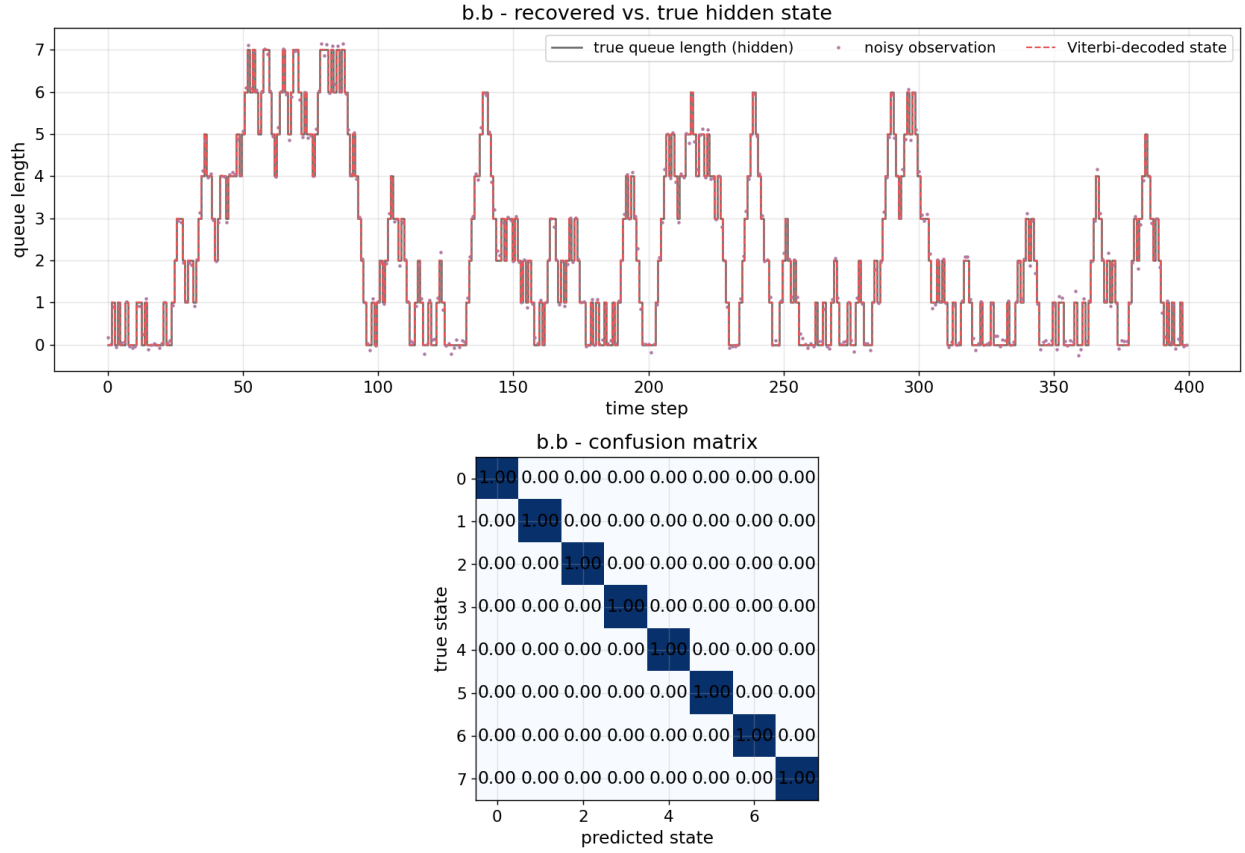


Fig 5 (b.b). Recovered vs. true hidden queue length (top) and confusion matrix (bottom).

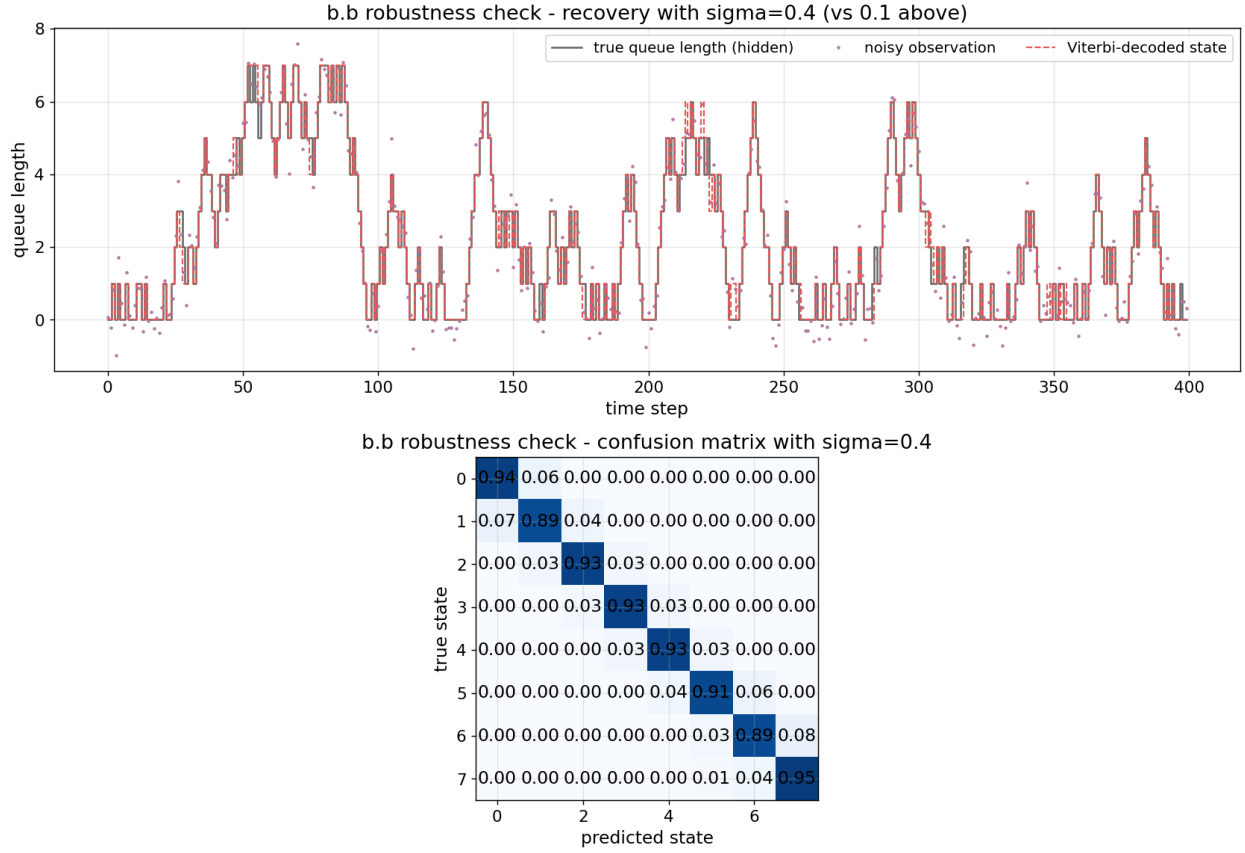


Fig 7 (b.b). Robustness check: same chain, noisier sensor ($\sigma = 0.4$ instead of 0.1) - recovery (top) and confusion matrix (bottom); accuracy drops from 100% to $\approx 92\%$.

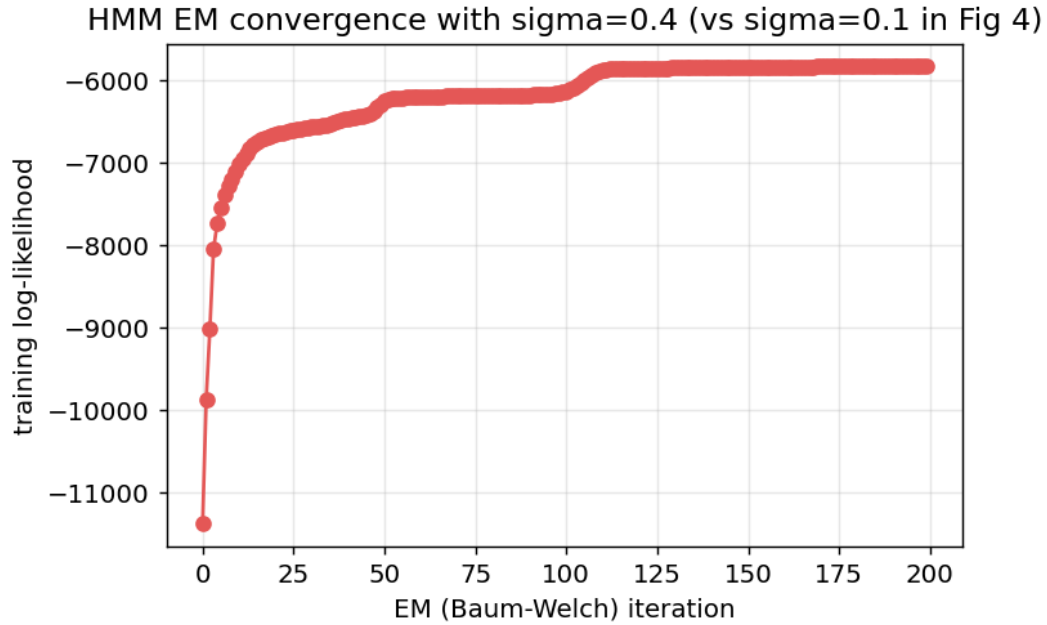


Fig 8 (b.b). EM convergence with $\sigma = 0.4$ (compare to Fig 4, $\sigma = 0.1$) - takes the full 200 iterations instead of 41 to converge.

1.2.1 Fit and EM convergence

We simulated data using the same λ and μ from part a) (the underlying chain dynamics), so the number of hidden states didn't need to be chosen separately - it is fixed at $K + 1 = 8$, matching the true chain. We compared different values of the observation noise σ to see how they affect recovery, and chose $\sigma = 0.1$ for the main analysis here (a larger $\sigma = 0.4$ is shown separately in b.b as a robustness check). We ran 20 random restarts (seeds) and picked the one with the highest training log-likelihood (Fig 4), since a single restart is unreliable here (see b.c) - different random seeds converge to very different results, sometimes with a large gap from the true state.

1.2.2 Quality of the approximation

Because the data is simulated, we have access to the ground truth - both the true hidden state at every time step and the true transition matrix P from part a) - which lets us directly score the HMM fit against it rather than relying only on likelihood. The table below compares the true and recovered emission means and self-transition probabilities $P(i, i)$; the recovered probabilities between states are almost identical to the true ones, with small deviations concentrated at the least-visited states.

state	true mean	recovered mean	true $P(i, i)$	recovered $P(i, i)$
0	0	-0.003	0.600	0.602
1	1	0.999	0.100	0.122
2	2	1.998	0.100	0.081
3	3	3.003	0.100	0.074
4	4	3.996	0.100	0.117
5	5	4.994	0.100	0.086
6	6	6.003	0.100	0.110
7	7	7.003	0.500	0.451

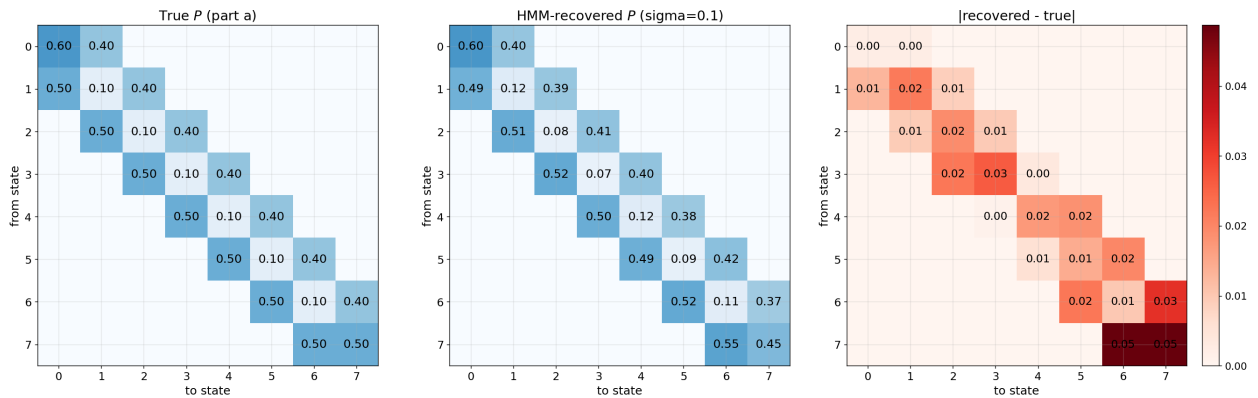


Fig 6 (b.b). True P (left), HMM-recovered P (middle), $\sigma = 0.1$ - the tridiagonal birth-death structure emerges without being imposed - and their absolute difference (right); the largest error (0.05) is at state 6-7, the least-visited transition.

Two separate questions are worth distinguishing here, and the figures above speak to both. The first is **state recovery**: the Viterbi-decoded state sequence (Fig 5, top) matches the true hidden

queue length almost exactly, with accuracy = 1.000 and a near-empty confusion matrix (Fig 5, bottom) - i.e. *which state the system was in at each time step* is recovered essentially perfectly, and held-out test log-likelihood/sample (-0.002) is close to training (0.030), so there is no gross overfitting. The second is **parameter recovery**: the recovered transition matrix (Fig 6, middle) is visually almost identical to the true one (Fig 6, left), reproducing the tridiagonal birth-death structure without it being imposed, but it is not exact - the absolute-difference heatmap (Fig 6, right) shows small residual errors (e.g. 0.074 vs. the true 0.1), concentrated at the state $6 \leftrightarrow 7$ boundary, the least-visited transition and therefore the one with the least data to estimate it from. So the approximation is excellent at the level of decoding the hidden path, and good but not exact at the level of the underlying transition probabilities. A caveat worth noting: the emission model is exactly correct by construction here (Gaussian noise generated the data), so this is a best-case demonstration - we checked robustness to a misspecified/noisier sensor separately below.

We re-ran the same best-of-20-restart procedure with $\sigma = 0.4$ instead of 0.1 (Figs 7-8). Accuracy drops from 100% to roughly 92%, and EM now needs the full 200 iterations to converge rather than 41 - a noisier sensor makes adjacent queue lengths harder to tell apart and makes the optimization itself slower, exactly as expected.

1.2.3 Sensitivity to initialization

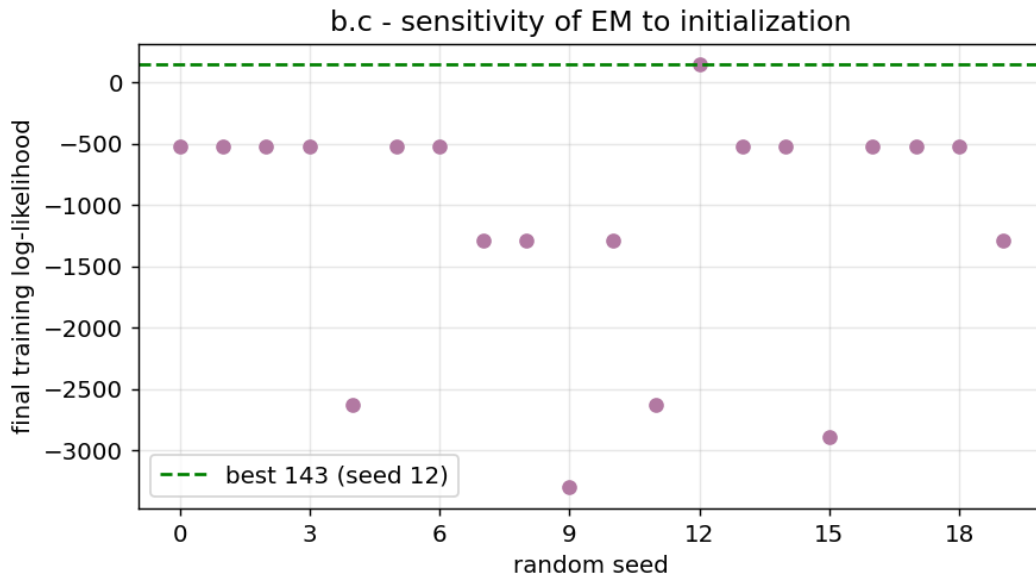


Fig 9 (b.c). Final log-likelihood across random initializations.

- **Seeds give different final log-likelihoods:** EM only moves uphill from wherever it starts, and the starting point is set by the random seed. Different seeds therefore climb to different final likelihood values, exactly the spread shown in Fig 9.
- **Non-convex likelihood $\Rightarrow$ only a local optimum:** the HMM log-likelihood surface has many peaks and valleys in parameter space. EM’s monotonicity guarantee only ensures the likelihood never decreases - it says nothing about reaching the global maximum, so a run can stop at any nearby peak.
- **Label-switching:** permuting the hidden-state labels (and their means/transitions together) leaves the likelihood unchanged, since the model has no fixed notion of “state 3”. This creates

many equivalent copies of the same good solution, which is why states are sorted by mean before any run-to-run comparison is made.

- **Flat regions:** rarely-visited states (e.g. the state 6 $\leftrightarrow$ 7 boundary in Fig 6) contribute little to the likelihood, so EM gets a weak gradient signal there and can stall on parameters tied to those states even while the rest of the fit is good.
- $\Rightarrow$ **multiple restarts, keep the best:** since any single run risks a poor local optimum, we run several random initializations and keep the one with the highest training log-likelihood - the one quantity directly comparable across runs - exactly the strategy used in b.a and validated empirically here.

1.3 Markov Chain Monte Carlo (25 pts)

Target: Bayesian logistic-regression posterior for a synthetic dataset (300 samples, 3 covariates plus intercept, generated from a known true β - no external dataset). Numerator cheap, normalizer $p(D)$ is an intractable d -dimensional integral. Sampled with from-scratch random-walk Metropolis-Hastings (acceptance ≈ 0.28 , near the 0.234 optimum).

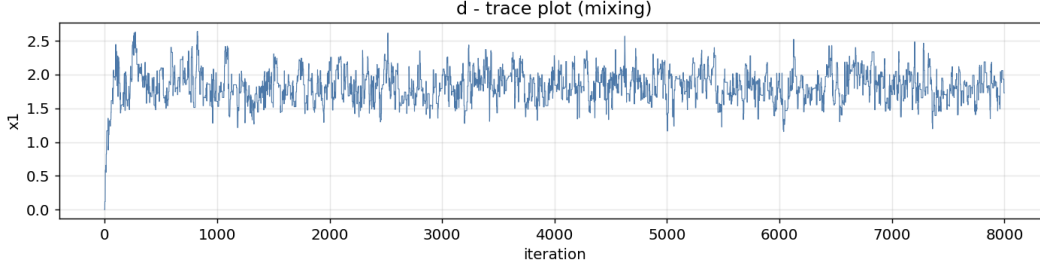


Fig 11 (d). Trace plot showing good mixing (support for convergence).

1.3.1 A distribution that is hard to sample from

- **Target:** we generate our own synthetic logistic-regression data : $n = 300$ samples with 3 covariates drawn i.i.d. $\mathcal{N}(0,1)$ plus an intercept, and $y_i \sim \text{Bernoulli}(\sigma(x_i^\top \beta_{\text{true}}))$ from a known, hand-picked β_{true} . The target is the Bayesian posterior $p(\beta \mid D)$ over the coefficients $\beta \in \mathbb{R}^d$ given this data, used to quantify uncertainty in the coefficients rather than just a point estimate. Prior: $\beta \sim \mathcal{N}(0, \tau^2 I)$; likelihood: logistic/Bernoulli, $y_i \sim \text{Bernoulli}(\sigma(x_i^\top \beta))$.
- **Why it's intractable:** $p(\beta \mid D) \propto p(D \mid \beta)p(\beta)$, but the normalizing constant (evidence) $p(D) = \int p(D \mid \beta)p(\beta) d\beta$ has no closed form, because no prior is conjugate to the logistic likelihood (unlike e.g. Gaussian likelihood + Gaussian prior, or Binomial + Beta).
- **Why not just compute it numerically:** approximating the integral on a grid over β -space costs $O(\text{res}^d)$ - exponential in the number of coefficients d - so a direct numerical approach is infeasible once d is more than a couple of dimensions (curse of dimensionality).
- **Consequence:** we can evaluate the unnormalized posterior (likelihood $\times$ prior) at any β , but cannot sample from it directly (no closed form to invert, no tractable normalizer for rejection sampling).

1.3.2 MCMC scheme and design choices

We sample the target $p(\beta \mid D)$ from d.a with a from-scratch random-walk Metropolis-Hastings chain. It is the right tool precisely because d.a showed we can evaluate the unnormalized posterior $p(D \mid \beta)p(\beta)$ at any β , but not the normalizer $p(D)$ - MH only ever needs *ratios* of the unnormalized density, so the intractable $p(D)$ cancels out. Algorithmically:

1. **Initialize:** start the chain at $\beta_0 = \mathbf{0}$ and compute the unnormalized log-posterior $\log(p(D \mid \beta_0)p(\beta_0))$.
2. **Propose:** at step t , draw $\beta' = \beta_t + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \text{step}^2 I)$ - a symmetric Gaussian random-walk proposal, so $q(\beta_t \rightarrow \beta') = q(\beta' \rightarrow \beta_t)$ and the Hastings correction is 1.
3. **Accept/reject:** compute the log-ratio of unnormalized posteriors, $\log p(D \mid \beta')p(\beta') - \log p(D \mid \beta_t)p(\beta_t)$ (the intractable $p(D)$ cancels); accept $\beta_{t+1} = \beta'$ with probability $\min(1, \text{ratio})$, else $\beta_{t+1} = \beta_t$.

4. **Tune the proposal scale:** the step size step is chosen so the empirical acceptance rate lands $\approx 0.2-0.4$ (near the 0.234 theoretical optimum) - too large and almost every proposal is rejected, too small and the chain barely moves.
5. **Burn-in:** discard the first samples (the chain started at $\beta_0 = 0$, not a draw from the posterior) so the retained samples no longer depend on the arbitrary start point.
6. **Thin:** keep only every k -th remaining sample to reduce the autocorrelation inherent to a random walk.
7. **Why this targets $p(\beta \mid D)$:** the accept/reject rule above satisfies detailed balance with respect to $p(\beta \mid D)$ by construction $\Rightarrow$ its stationary distribution is exactly the target posterior, so the retained samples (after burn-in and thinning) approximate draws from $p(\beta \mid D)$.

1.3.3 Histograms demonstrating the target is reached

Running the scheme above (step size tuned to acceptance ≈ 0.28 , near the 0.234 optimum) for 40,000 iterations, discarding burn-in and thinning, gives the per-coefficient posterior samples below:

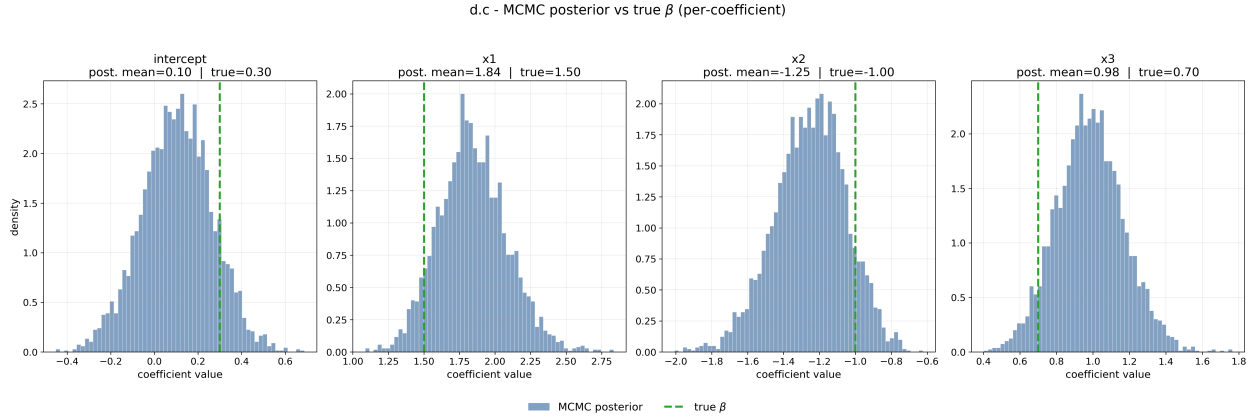


Fig 10 (d.c). MCMC posterior histograms vs the known true β (dashed green) - available directly since the data is synthetic.

- Because the data is synthetic we have the true β to check against directly (no need for a separate reference approximation), and it falls within the posterior's bulk for every coefficient; trace mixes; Gelman-Rubin $\hat{R} < 1.1$ for all parameters $\Rightarrow$ target reached.

2 Martingales

GBSG2 (German Breast Cancer Study Group 2) is real censored survival data from 686 breast-cancer patients: for each patient we observe either the time to disease recurrence (or death) or, if the patient was still event-free when the study ended or they were lost to follow-up, the time at which observation stopped (censoring). This is a genuine real-life random process: whether and when an individual patient's cancer recurs depends on a mix of biological variability and patient-specific factors that are never fully observed or measurable, so the recurrence/death time is inherently random rather than a deterministic function of the recorded covariates.

This process is not itself a martingale, but is analyzed using one. The real-life process here - the cumulative number of recurrences/deaths observed by time n , N_n - is *non-decreasing*, so it cannot have conditionally mean-zero increments: it is not a martingale. Just as the waiting time for a pattern is not itself a martingale but an auxiliary martingale is constructed to analyze it, here we construct an auxiliary process from N_n and use it to say something about N_n itself. Concretely, by the Doob-Meyer decomposition, N_n splits into a *predictable compensator* A_n (the expected accumulation so far, given everything known up to now, built from the hazard and the number of patients still at risk) plus a leftover term $M_n = N_n - A_n$, and it is this constructed M_n - not N_n - that is a genuine martingale: at each step, N_n can only move by the amount A_n already “predicts” on average, so what is left over is unpredictable given the past. This is possible because the risk set and hazard at each step are known before that step's outcome is observed - exactly the predictability Doob-Meyer requires. We then use this constructed martingale M_n to derive a property of the original, non-martingale process N_n : the unbiasedness of the Nelson-Aalen cumulative-hazard estimator.

2.1 Martingale Modeling (a.a + a.b, 20 pts)

Process: the counting-process (Doob-Meyer / Nelson-Aalen) martingale $M_n = N_n - A_n$ on GBSG2, where N_n = cumulative events by year n and A_n = compensator $\sum(\# \text{at risk}) \cdot h$. Increments are conditionally mean-zero $\Rightarrow$ martingale.

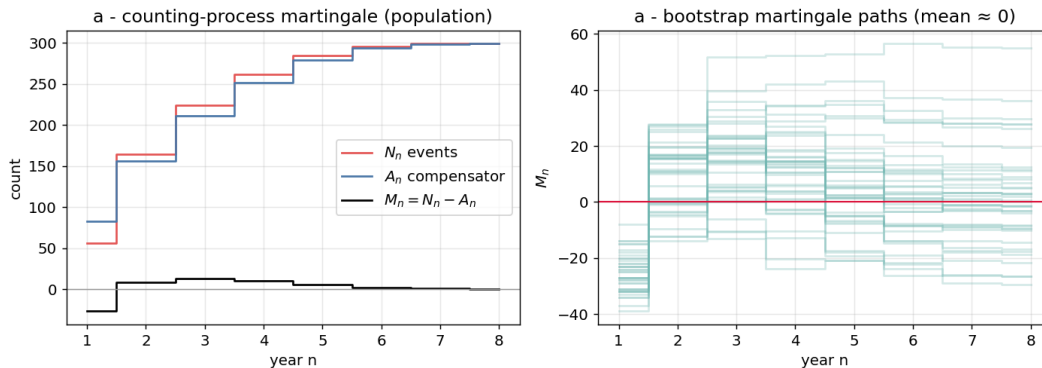


Fig 12. Counting process, compensator and martingale (left); bootstrap martingale paths, mean ≈ 0 (right).

2.1.1 Why it is a good / bad martingale model

As established above, N_n itself is not a martingale; the discussion below is about the constructed martingale $M_n = N_n - A_n$ used to analyze it - meaning we are checking whether this is a good or bad *analysis tool* for the real process N_n .

Why it is a good martingale model:

1. The compensator A_n is *predictable* - it is built from the hazard h and the number of patients still at risk, both known just before step n , so it represents the “expected events so far” computed only from information already available, never from the outcome it is trying to predict.
2. By the Doob-Meyer decomposition, $M_n = N_n - A_n$ therefore has conditionally mean-zero increments by construction: given everything known up to now, the next increment of M_n is equally likely to be positive or negative - the defining “fair game” property of a martingale.
3. M_n is exactly the *running surprise* - the part of the observed event count not already explained by the hazard model - which cleanly isolates whatever randomness the model does not account for from the part it does.
4. This construction is not ad hoc: it is the same martingale structure underlying standard, well-established survival-analysis tools (the Nelson-Aalen cumulative-hazard estimator, the log-rank test), so the modeling choice is backed by established theory, not invented for this report.

Why it is a bad (or only approximate) martingale model:

1. It requires a *correctly specified* hazard h . If the true instantaneous risk depends on unobserved patient heterogeneity (a “frailty” effect) or on covariates not folded into the pooled hazard, the compensator A_n is systematically biased, and M_n drifts rather than fluctuating around zero - i.e. it is then only *approximately* a martingale.
2. Discretizing continuous follow-up time into 1-year bins can artificially create *ties* (multiple events landing in the same bin) that the idealized continuous-time counting-process assumptions treat as probability zero; properly handling ties (e.g. Breslow/Efron corrections) is not done by this simple pooled-hazard model.
3. If the true hazard is *time-varying* in a way the (coarse, pooled or binned) hazard estimate does not capture, the compensator will over- or under-predict events in specific periods, again producing drift rather than a clean mean-zero residual.
4. Because these are systematic biases rather than large one-off errors, they can be hard to distinguish from ordinary sampling noise in a single dataset - detecting them typically needs either many patients or a resampling/bootstrap check (as done on the right panel of Fig 12).

2.1.2 Stopping time and Optional Stopping Theorem

1. **Define the stopping time.** Let τ be the first year the patient leaves the event-free state - either because the event (recurrence/death) occurs, or because follow-up ends (administrative censoring). Whether $\tau = n$ depends only on what has been observed by year n (event/censoring status up to n), so τ is a genuine stopping time with respect to the same filtration M_n is adapted to.
2. **Check an OST condition.** The Optional Stopping Theorem applies provided at least one of a few sufficient conditions holds. Here the follow-up is discretized into a finite grid of n_{bins} year-bins, so every patient’s τ satisfies $\tau \leq n_{\text{bins}}$ with probability 1 - i.e. τ is a *bounded*

stopping time. This is the easiest condition to verify here: we do not need to separately check integrability of τ or boundedness of M_n 's increments - boundedness of the time grid alone suffices.

3. **Result.** $M_n = N_n - A_n$ is a martingale (a.a) and τ is a bounded stopping time, so the Optional Stopping Theorem gives

$$\mathbb{E}[M_\tau] = \mathbb{E}[M_0] = 0.$$

($M_0 = N_0 - A_0 = 0$ trivially: no events and no compensator have accumulated before year 0.)

4. **What this means.** $\mathbb{E}[M_\tau] = 0$ means $\mathbb{E}[N_\tau] = \mathbb{E}[A_\tau]$. No matter when a patient's outcome resolves - through the event or through (non-informative) censoring - the model's predictions have no systematic bias, on average across patients. The interesting part: a fair game cannot be beaten by choosing when to stop, and here "stopping" is each patient's own outcome time - so the prediction stays unbiased in aggregate even though every patient is followed for a different, random length of time.

2.2 Stochastic Gradient Descent (30 pts)

Scheme: Nesterov accelerated SGD. The velocity update is $v_{t+1} = \mu v_t + g_t$, where g_t is a stochastic gradient taken at the look-ahead point $y_t = x_t - \eta_t \mu v_t$, and the parameter update is $x_{t+1} = x_t - \eta_t v_{t+1}$. Step size follows a decreasing Robbins-Monro schedule $\eta_t = \eta_0 / (1 + t/t_0)$. Tested on the convex logistic loss from the synthetic data in part d), compared against vanilla SGD over 20 random seeds.

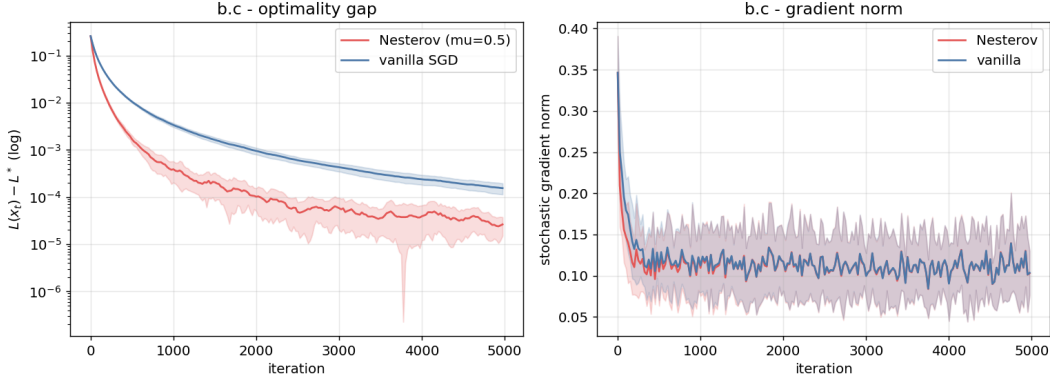


Fig 13 (b.c). Optimality gap $L(x_t) - L^*$ (log scale) and gradient norm, mean $\pm$ std over 20 seeds.

2.2.1 The scheme and its motivation

Answer - b.a

1. **Velocity acts as a running average.** v_t builds up from past gradients through $v_{t+1} = \mu v_t + g_t$, so it behaves like an average over many steps rather than a reaction to one noisy sample. This keeps the descent direction steady across iterations.
2. **What makes it Nesterov instead of heavy-ball.** The gradient is computed at $y_t = x_t - \eta_t \mu v_t$, the point momentum is already carrying the iterate toward, rather than at the current point x_t . Heavy-ball momentum only ever evaluates the gradient at the current position.
3. **Why the look-ahead helps.** Evaluating at y_t means the step already reflects what the loss looks like after the momentum move, not before it. If momentum is about to push past a bend in the loss surface, that shows up in the gradient at y_t , so the step pulls back early instead of correcting only after the overshoot has happened.
4. **Net effect.** Nesterov keeps the noise-smoothing benefit of momentum and adds this early correction on top, which is why it converges faster in Fig 13 with no extra sampling cost. Only the point where the gradient is evaluated changes, not how many samples each step uses.

2.2.2 Convergence proof (based on class tools)

Answer - b.b

1. **Assumptions.** L is convex and ℓ -smooth (∇L is ℓ -Lipschitz), with minimizer x^* . Let $\mathcal{F}_t = \sigma(x_0, \dots, x_t, v_0, \dots, v_t)$. The gradient g_t is unbiased for $\nabla L(y_t)$ given $\mathcal{F}_t$, with $\mathbb{E}\|g_t\|^2 \leq G^2$.
2. **Potential function.** Define $X_t = \|x_t - x^*\|^2 + c\|v_t\|^2$ for a constant $c > 0$, chosen so the momentum term enters with the right sign in the drift bound below.

3. **Connecting y_t back to x_t .** Convexity at y_t gives $L(y_t) - L^* \leq \langle \nabla L(y_t), y_t - x^* \rangle$, and smoothness gives

$$L(x_t) - L(y_t) \leq \langle \nabla L(y_t), x_t - y_t \rangle + \frac{\ell}{2} \|x_t - y_t\|^2 = \langle \nabla L(y_t), \eta_t \mu v_t \rangle + \frac{\ell}{2} \eta_t^2 \mu^2 \|v_t\|^2.$$

Adding these two gives

$$L(x_t) - L^* \leq \langle \nabla L(y_t), x_t - x^* \rangle + \frac{\ell}{2} \eta_t^2 \mu^2 \|v_t\|^2.$$

This is the one step added beyond the standard momentum proof from class. The look-ahead evaluation only costs an $O(\eta_t^2)$ term, not a first-order one, so the rest of the argument carries through unchanged.

4. **One-step drift.** Expand $\|x_{t+1} - x^*\|^2$ using $x_{t+1} = x_t - \eta_t v_{t+1}$, take $\mathbb{E}[\cdot \mid \mathcal{F}_t]$, and combine with the bound above to get

$$\mathbb{E}[X_{t+1} \mid \mathcal{F}_t] \leq X_t(1 + \beta_t) + \xi_t - \zeta_t, \quad \zeta_t = 2\eta_t(L(x_t) - L^*) \geq 0.$$

The smoothness correction from the previous step is folded entirely into $\beta_t, \xi_t = O(\eta_t^2)$, so it only adds a small perturbation and does not weaken the descent term ζ_t .

5. **Summability.** Since $\sum \eta_t^2 < \infty$ under the Robbins-Monro schedule, both $\sum \xi_t < \infty$ and $\sum \beta_t < \infty$ follow.
6. **Robbins-Siegmund theorem.** A nonnegative process X_t with summable β_t, ξ_t satisfies, by Robbins-Siegmund (Martingales lecture, slide 70), $X_t \rightarrow X_\infty < \infty$ a.s., and $\sum_t \zeta_t < \infty$ a.s.
7. **Putting it together.** Since $\zeta_t \geq 0$ has a finite sum while $\sum \eta_t = \infty$, it follows that $\liminf_t (L(x_t) - L^*) = 0$. Because X_t converges, x_t stays bounded, so the liminf upgrades to a full limit: $L(x_t) \rightarrow L^*$ along the whole sequence. This matches the structure of the class SGD proof (slides 80-82), with the momentum and look-ahead terms folded into the potential and drift rather than requiring a new argument.

2.2.3 Convergence over many realizations

Answer - b.c

1. **Optimality gap (Fig 13, left).** For both schemes, $L(x_t) - L^*$ decays toward 0 across all 20 seeds, with tight bands around the mean, so the behavior is consistent rather than seed-dependent. Nesterov's curve stays below vanilla SGD's throughout, meaning it reaches any given gap in fewer iterations, not only at the end.
2. **Final values.** With $L^* = 0.4304$, Nesterov finishes at 0.4305 and vanilla SGD at 0.4306. Both are close to optimal since the problem is low-dimensional and well-conditioned, but Nesterov stays the tighter of the two throughout, matching the faster descent predicted in b.b.
3. **Gradient norm (Fig 13, right).** Both schemes' gradient norm shrinks and settles at a similar floor, which mainly reflects the shared minibatch noise rather than a real difference between the two methods.
4. **Link to the proof.** The gap converging to 0 while $\sum \zeta_t < \infty$ is exactly what b.b predicts. Since Nesterov's look-ahead correction is only an $O(\eta_t^2)$ term, both methods must converge, the correction only changes how fast they get there.